

[← Go Back](#)

Hardware

Portenta Vision Shield

Tutorials

[Blob Detection with Portenta and OpenMV](#)[Saving Bitmap Camera Images to the SD Card](#)[Connect Portenta Vision Shield to TTN with LoRa® Technology](#)[Creating a Basic Face Filter With OpenMV](#)[Training a Custom Machine Learning Model for Portenta H7](#)[Using the Portenta Vision Shield - Ethernet with Arduino IDE](#)[Using the Portenta Vision Shield - Ethernet with OpenMV](#)[Getting Started With the Portenta Vision Shield Camera](#)[Voice Commands With The Arduino Speech Recognition Engine](#)[Connecting to The Things Network Using OpenMV](#)

Portenta Vision Shield User Manual

Home / Hardware / Portenta Vision Shield / **Portenta Vision Shield User Manual**

Portenta Vision Shield User Manual

Learn about the hardware and software features of the Arduino® Portenta Vision Shield.

Author · Christopher Méndez · Last revision · 02/28/2025

Overview

This user manual will guide you through a practical journey covering the most interesting features of the Arduino Portenta Vision Shield. With this user manual, you will learn how to set up, configure and use this Arduino board.

Hardware and Software Requirements

Hardware Requirements

Portenta Vision Shield Ethernet (x1) or **Portenta Vision Shield with LoRa® technology**

Portenta H7 (x1) or **Portenta C33** (x1)

USB-C® cable cable (x1)

Software Requirements

ON THIS PAGE

Overview

Hardware and Software Requirements	+
Product Overview	+
First Use	+
Camera	+
Microphone	+
Machine Learning Tool	+
Ethernet (ASX00021)	
LoRa® (ASX00026)	
Support	+
Trademark Acknowledgments	

[Help](#)

Arduino IDE 1.8.10+, Arduino IDE 2.0+, or Arduino Web Editor

To create custom Machine Learning models, the Machine Learning Tools add-on integrated into the [Arduino Cloud](#) is needed. In case you do not have an Arduino Cloud account, you will need to create one first.

Product Overview

The Arduino Portenta Vision Shield is an add-on board providing machine vision capabilities and additional connectivity to the Portenta family of Arduino boards, designed to meet the needs of industrial automation. The Portenta Vision Shield connects via a high-density connector to the Portenta boards with minimal hardware and software setup.

The Portenta Vision Shield has two hardware revisions, distinguished only by the  onboard camera sensor; all other features of the shield remain identical across both revisions.

The Portenta Vision Shield (**Rev 1**) includes the **HM01B0** (1/11" 320 x 320 VGA 60FPS) CMOS camera module.

The Portenta Vision Shield (**Rev 2**) includes the **HM0360** (1/6" 640 x 480 VGA 60FPS) CMOS camera module.

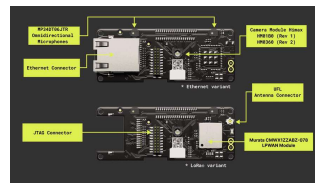
The included camera module has been pre-configured to work with the OpenMV libraries provided by Arduino. Based on the specific application requirements, the

two configurations with either Ethernet or LoRa® connectivity.

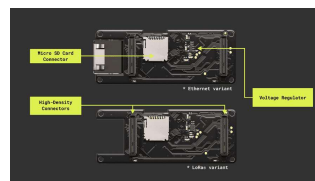
Board Architecture

Overview

The Portenta Vision Shield with LoRa® technology brings industry-rated features to your Portenta. This hardware add-on will let you run embedded computer vision applications, connect wirelessly via LoRa technology to the Arduino Cloud or your own infrastructure, and activate your system upon the detection of sound events.



Vision Shield main components (top view)



Vision Shield main components (bottom view)

Here is an overview of the board's main components, as shown in the images above:

Power Regulator: the Portenta H7/C33 supplies 3.3 V power to the LoRa® module (ASX00026 only), Ethernet communication (ASX00021 only), Micro SD slot and dual microphones via the 3.3 V output of the high-density connectors. An onboard LDO regulator

Camera: the Himax HM01B0 (320x320) and HM0360 (640x480) modules are very low-power cameras with a maximum of 60 FPS depending on the operating mode. Video data is transferred over a configurable 8-bit interface with support for frame and line synchronization. The module delivered with the Portenta Vision Shield is the monochrome version. Configuration is achieved via an I2C connection with the compatible Portenta boards microcontrollers.

Himax modules offers very low-power image acquisition and provides the possibility to perform motion detection without main processor interaction. The “Always-on” operation provides the ability to turn on the main processor when movement is detected with minimal power consumption.



The Portenta C33 is not compatible with the camera of the Portenta Vision Shield

Digital Microphones: the dual MP34DT05 digital MEMS microphones are omnidirectional and operate via a capacitive sensing element with a high (64 dB) signal-to-noise ratio. The microphones have been configured to provide separate left and right audio over a single PDM stream.

The sensing element, capable of detecting acoustic waves, is manufactured using a specialized silicon micromachining process dedicated to produce audio sensors.

Micro SD Card Slot: a Micro SD card slot is available under the Portenta Vision Shield board. Available libraries allow reading and writing to FAT16/32 formatted cards

Ethernet (ASX00021 Only): ethernet connector allows connecting to 10/100 Base TX networks using the Ethernet PHY available on the Portenta board.

LoRa® Module (ASX00026 Only): LoRa® connectivity is provided by the Murata CMWX1ZZABZ module. This module contains an STM32L0 processor along with a Semtech SX1276 Radio. The processor is running on Arduino open-source firmware based on Semtech code.

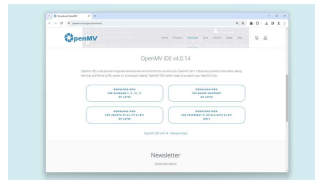
Shield Environment Setup

Connect the Vision Shield with a Portenta H7 through their High-Density connectors and verify they are correctly aligned.



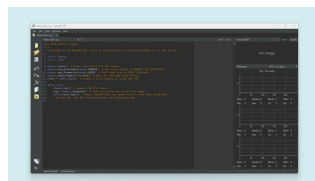
Before you can start programming MicroPython scripts for the Vision Shield, you need to download and install the OpenMV IDE.

Open the [OpenMV](#) download page in your browser, download the latest version available for your operating system, and follow the instructions of the installer.



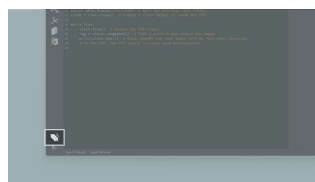
OpenMV Download Page

Open the **OpenMV IDE** and connect the Portenta H7 to your computer via the USB cable if you have not done so yet.



The OpenMV IDE after starting it

Click on the "connect" symbol at the bottom of the left toolbar.



Click the connect button to attach the Portenta H7 to the OpenMV IDE

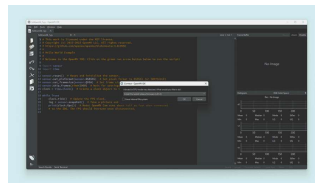
If your Portenta H7 does not have the latest firmware, a pop-up will ask you

DFU mode and its green LED will start fading.

Select

Install the latest [release](#) firmware

. This will install the latest OpenMV firmware on the H7. You can leave the option of erasing the internal file system unselected and click



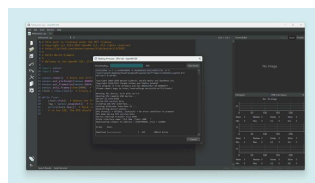
Install the latest version of the OpenMV firmware

Portenta H7's green LED will start flashing while the OpenMV firmware is being uploaded to the board. A loading bar will start showing you the flashing progress.

Wait until the green LED stops flashing and fading. You will see a message saying

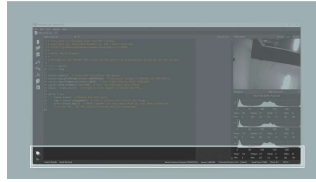
DFU firmware update complete!

when the process is done.



Installing firmware on H7 board in OpenMV

The board will start flashing its blue LED when it is ready to be connected. After confirming the completion dialog, the Portenta H7 should already be connected to the OpenMV IDE, otherwise, click the "connect" button (plug symbol) once again (the



When the H7 is successfully connected a green play button appears

While using the Portenta H7 with OpenMV, the RGB LED of the board can be used to inform the user about its current status. Some of the most important ones are the following:

● **Blinking Green:** Your Portenta H7 onboard bootloader is running. The onboard bootloader runs for a few seconds when your H7 is powered via USB to allow OpenMV IDE to reprogram your Portenta.

● **Blinking Blue:** Your Portenta H7 is running the default **main.py** script onboard.

If you overwrite the **main.py** script on your Portenta H7, then it will run whatever code you loaded on it instead.

If the LED is blinking blue but OpenMV IDE cannot connect to your Portenta H7, please make

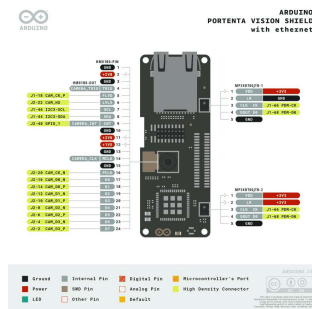
i sure you are connecting your Portenta H7 to your PC with a USB cable that supplies both data and power.

● **Blinking White:** Your Portenta H7 firmware is panicking because of a hardware failure. Please check that your Vision Shield's camera module is installed securely.

If you tap the Portenta H7 reset button once, the board resets. If you tap it twice, the board

i enters Device Firmware Upgrade (DFU) mode and its green LED starts blinking and fading.

Pinout



Vision Shield simple pinout

The full pinout is available and downloadable as PDF from the link below:

[Vision Shield full pinout](#)

Datasheet

The complete datasheet is available and downloadable as PDF from the link below:

[Vision Shield datasheet](#)

Schematics

The complete schematics are available and downloadable as PDF from the links below:

[Vision Shield - Ethernet schematics](#)

[Vision Shield - LoRa® schematics](#)

STEP Files

The complete STEP files are available and downloadable from the link below:

[Vision Shield STEP files](#)

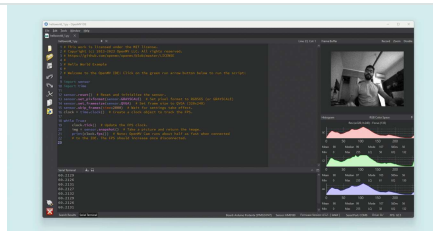
First Use

Hello World Example

Working with camera modules, the `Hello World` classic example is not an LED blink but the simplest sketch to capture images. We will use this example to verify the board's connection to the IDEs and that the Vision Shield itself is working as expected.

The following example script can be found on **File > Examples > HelloWorld > helloworld.py** in the OpenMV IDE.

```
1 import sensor
2 import time
3
4 sensor.reset() # Reset a
5 sensor.set_pixformat(sens
6 sensor.set_framesize(sens
7 sensor.skip_frames(time=2
8 clock = time.clock() # C
9
10 while True:
11     clock.tick() # Updat
12     img = sensor.snapshot
13     print(clock.fps()) #
14     # to the IDE. The FPS
```



From the above example script, we can highlight the main functions:

```
sensor.set_pixformat(<Sensor>)
```

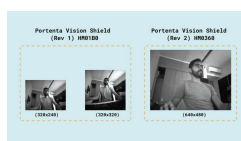
lets you set the pixel format for the camera sensor. The Vision Shield is compatible with these: `sensor.GRAYSCALE`, and `sensor.BAYER`.

To define the pixel format to any of the supported ones, just add it to the `set_pixformat` function argument.

```
sensor.set_framesize(<Resolution>)
```

lets you define the image frame size in terms of pixels.

[Here](#) you can find all the different options.



Different resolutions examples

Here are some tested settings for your camera setup:

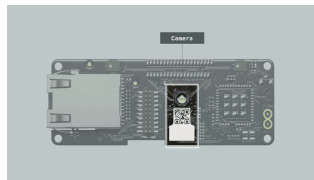
Resolution	Setting	Compatibility	Note
320 x 240	<code>sensor.QVGA</code>	HM01 B0 and HM03 60	

Res olu tio n	Setting	Comp atibili ty	Note
320 x 320	<code>sensor.B320X320</code>	HM01B0	Full sensor resolution
640 x 480	<code>sensor.VGA</code>	HM0360	Full sensor resolution

`sensor.snapshot()` lets you take a picture and return the image so you can save it, stream it or process it.

Camera

The Portenta Vision Shields's main feature is its onboard camera, based on the HM01B0 or HM0360 ultra low power CMOS image sensor. It is perfect for Machine Learning applications such as object detection, image classification, machine/computer vision, robotics, IoT, and more.



Onboard camera sensor

HM01B0 Camera Features

Ultra-Low-Power Image Sensor designed for always-on vision

High-sensitivity 3.6 μ
BrightSense™ pixel technology
Window, vertical flip and
horizontal mirror readout
Programmable black level
calibration target, frame size,
frame rate, exposure, analog
gain (up to 8x) and digital gain
(up to 4x)
Automatic exposure and gain
control loop with support for 50
Hz / 60 Hz flicker avoidance
Motion Detection circuit with
programmable ROI and
detection threshold with digital
output to serve as an interrupt

Supported Resolutions

QQVGA (160x120) at 15, 30, and
60 FPS

QVGA (320x240) at 15, 30 and
60 FPS

B320X320 (320x320) at 15, 30
and 45 FPS

Power Consumption

< 1.1 mW QQVGA resolution at
30 FPS

< 2 mW QVGA resolution at 30
FPS

< 4 mW QVGA resolution at 60
FPS

HM0360 Camera Features

Ultra-Low-Power, high
sensitivity, low noise VGA
sensor

On-chip auto exposure / gain
and zone detection

Automatic wake and sleep
operation with programmable
event interrupt to host
processor

Pre-metered exposure provides
well exposed first frame and
after extended sleep (blanking)
period

Embedded line sensor

other interrupt event
information

Supported Resolutions

QQVGA (160x120) at 15, 30, and
60 FPS

QVGA (320x240) at 15, 30 and
60 FPS

VGA (640x480) at 15, 30 and 60
FPS

Power Consumption

140 μ A QVGA resolution at 2
FPS

3.2 mA QVGA resolution at 60
FPS

7.8 mA VGA resolution at 60 FPS

The Vision Shield is primarily
intended to be used with the
OpenMV MicroPython ecosystem. So,
it's recommended to use this IDE for
machine vision applications.

Snapshot Example

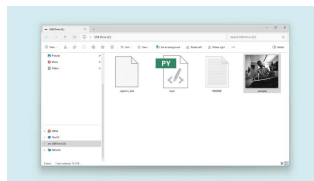
The example code below lets you
take a picture and save it on the
Portenta H7 local storage or in a
Micro SD card as `example.jpg`.



```
1 import sensor
2 import time
3 import machine
4
5 sensor.reset() # Reset a
6 sensor.set_pixformat(sens
7 sensor.set_framesize(sens
8 sensor.skip_frames(time=2
9
10 led = machine.LED("LED_BL
11
12 start = time.ticks_ms()
13 while time.ticks_diff(tim
14     sensor.snapshot()
15     led.toggle()
16
17 led.off()
18
19 img = sensor.snapshot()
20 img.save("example.jpg")
21
22 raise (Exception("Please
```

i If a Micro SD card is inserted into the Vision Shield, the snapshot will be stored there

After the snapshot is taken, reset the board by pressing the reset button and the image will be on the board storage drive.



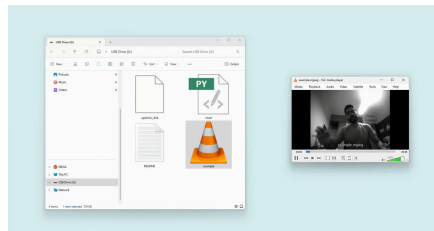
Snapshot saved in H7 local storage

Video Recording Example

The example code below lets you record a video and save it on the Portenta H7 local storage or in a Micro SD card as `example.mjpeg`.

```
1 import sensor
2 import time
3 import mjpeg
4 import machine
5
6 sensor.reset() # Reset a
7 sensor.set_pixformat(sens
8 sensor.set_framesize(sens
9 sensor.skip_frames(time=2
10
11 led = machine.LED("LED_RE
12
13 led.on()
14 m = mjpeg.Mjpeg("example.
15
16 clock = time.clock() # C
17 for i in range(50):
18     clock.tick()
19     m.add_frame(sensor.sn
20     print(clock.fps())
21
22 m.close()
23 led.off()
24
25 raise (Exception("Please
```

We recommend you use [VLC](#) to play the video.



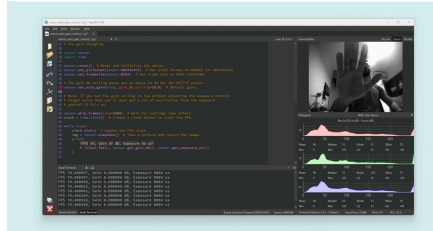
Sensor Control

There are several functions that allow us to configure the behavior of the camera sensor and adapt it to our needs.

Gain: the gain is related to the sensor sensitivity and affects how bright or dark the final image will be.

With the following functions, you can control the camera gain:


```
1 sensor.set_auto_gain(True,  
2 sensor.set_auto_gain(False
```



Orientation: flip the image captured to meet your application's needs.

With the following functions, you can control the image orientation:

```
1 sensor.set_hmirror(True) #  
2 sensor.set_vflip(True) # E
```

You can find complete

Sensor Control examples in **File >**

Examples > Camera > Sensor

Control of the OpenMV IDE.

Bar and QR Codes

The Vision Shield is ideal for production line inspections, in these examples, we are going to be locating and reading bar codes and QR codes.

Bar Codes

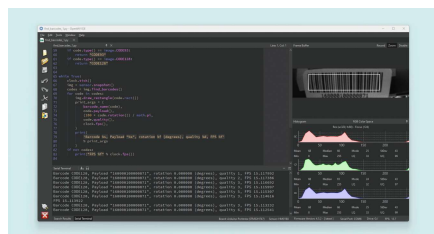
This example code can be found in **File > Examples > Barcodes** in the OpenMV IDE.

```

1 import sensor
2 import image
3 import time
4 import math
5
6 sensor.reset()
7 sensor.set_pixformat(sensor.RGB565)
8 sensor.set_framesize(sensor.QQ320)
9 sensor.set_windowing((64, 64))
10 sensor.skip_frames(time=2000)
11 sensor.set_auto_gain(False)
12 sensor.set_auto_whitebal(False)
13 clock = time.clock()
14
15 # Barcode detection can
16
17 def barcode_name(code):
18     if code.type() == image.COD_1D:
19         return "EAN2"
20     if code.type() == image.COD_1D:
21         return "EAN5"
22     if code.type() == image.COD_1D:
23         return "EAN8"
24     if code.type() == image.COD_1D:
25         return "UPCE"
26     if code.type() == image.COD_1D:
27         return "ISBN10"
28     if code.type() == image.COD_1D:
29         return "ISBN13"

```

The format, payload, orientation and quality will be printed out in the Serial Monitor when a bar code becomes readable.

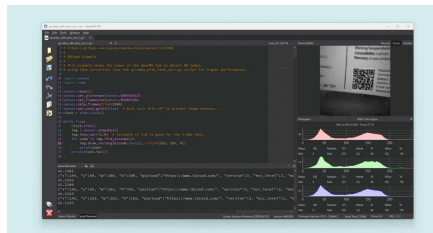


QR Codes

This example code can be found in **File > Examples > Barcodes** in the OpenMV IDE.

```
1 import sensor
2 import time
3
4 sensor.reset()
5 sensor.set_pixformat(sensor.RGB565)
6 sensor.set_framesize(sensor.VGA)
7 sensor.skip_frames(time=2000)
8 sensor.set_auto_gain(False)
9 clock = time.clock()
10
11 while True:
12     clock.tick()
13     img = sensor.snapshot()
14     img.lens_corr(1.8)
15     for code in img.find_qr_codes():
16         img.draw_rectangle(code.rect())
17         print(code)
18     print(clock.fps())
```

The coordinates, size, and payload will be printed out in the Serial Monitor when a QR code becomes readable.



Face Tracking

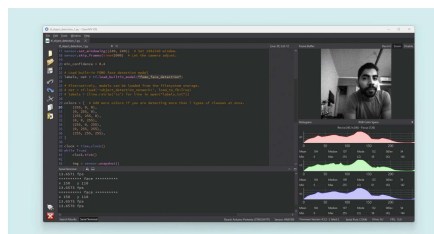
You can track faces using the built-in FOMO face detection model. This example can be found in **File > Examples > Machine Learning > TensorFlow > tf_object_detection.py**.

This script will draw a circle on each detected face and will print their coordinates in the Serial Monitor.

```

1 import sensor
2 import time
3 import ml
4 from ml.utils import NMS
5 import math
6 import image
7
8 sensor.reset() # Reset
9 sensor.set_pixformat(sensor.RGB565)
10 sensor.set_framesize(sensor.QQVGA)
11 sensor.set_windowing((240, 240))
12 sensor.skip_frames(time=2000)
13
14 min_confidence = 0.4
15 threshold_list = [(math.pi/2, 0.5),
16                   (math.pi/4, 0.5),
17                   (math.pi/8, 0.5)]
18 # Load built-in FOMO face model
19 model = ml.Model("fomo_tflite")
20 print(model)
21
22 # Alternatively, models can be loaded from a file
23 # model = ml.Model('object.tflite')
24 # labels = [line.rstrip() for line in open('object.txt')]
25
26 colors = [ # Add more colors here
27           (255, 0, 0),
28           (0, 255, 0),
29           (255, 255, 0),
30           (0, 0, 255),
31           (255, 0, 255),
32           (0, 255, 255)]

```



You can load different **Machine Learning** models for detecting other objects, for example, persons.

Download the `.tflite` and `.txt` files from this [repository](#) and copy them to the Portenta H7 local storage.

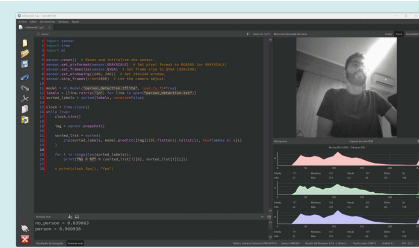
Use the following example script to run the **person detection** model.

```

1  import sensor
2  import time
3  import ml
4
5  sensor.reset() # Reset a
6  sensor.set_pixformat(sens
7  sensor.set_framesize(sens
8  sensor.set_windowing((240
9  sensor.skip_frames(time=2
10
11  model = ml.Model('person_
12  labels = [line.rstrip('\n
13  sorted_labels = sorted(la
14
15  clock = time.clock()
16  while True:
17      clock.tick()
18
19      img = sensor.snapshot
20
21      sorted_list = sorted(
22          zip(sorted_labels
23      )
24      for i in range(len(so
25          print("%s = %f" %
26
27      print(clock.fps(), "f

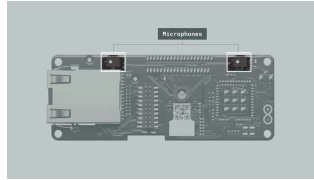
```

When a person is in the field of view of the camera, you should see the inference result for `person` rising above 70% of certainty.



Microphone

The Portenta Vision Shield features two omnidirectional microphones, based on the MP34DT05 ultra-compact, low-power, and digital MEMS microphone.



Vision Shield
omnidirectional
microphones

Features:

AOP = 122.5 dB SPL

64 dB signal-to-noise ratio

Omnidirectional sensitivity

-26 dBFS $\pm$ 1 dB sensitivity

FFT Example

You can analyze frequencies present in sounds alongside their harmonic features using this example.

By measuring the sound level on each microphone we can easily know from where the sound is coming, an interesting capability for robotics and AIoT applications.

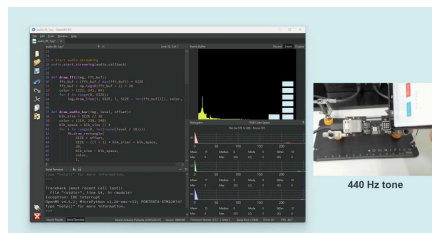


```

1 import image
2 import audio
3 from ulab import numpy as np
4 from ulab import utils
5
6 CHANNELS = 2
7 SIZE = 512 // (2 * CHANNELS)
8
9 raw_buf = None
10 fb = image.Image(SIZE + SIZE)
11 audio.init(channels=CHANNELS, size=SIZE)
12
13
14 def audio_callback(buf)
15     # NOTE: do Not call
16     global raw_buf
17     if raw_buf is None:
18         raw_buf = buf
19
20
21 # Start audio streaming
22 audio.start_streaming(audio_callback)
23
24
25 def draw_fft(img, fft_buf)
26     fft_buf = (fft_buf, fft_buf)
27     fft_buf = np.log10(fft_buf)
28     color = (222, 241, 255)

```

With this script running you will be able to see the Fast Fourier Transform result in the image viewport. Also, the sound level on each microphone channel.



Speech Recognition Example

You can easily implement sound/voice recognition applications using Machine Learning on the edge, this means that the Portenta H7 plus

Use the following script to run the example. It can also be found on **File > Examples > Audio > micro_speech.py** in the OpenMV IDE.

```
1 import time
2 from ml.apps import Micro
3
4
5 def callback(label, score
6     print(f'\nHeard: "{la
7
8
9 # By default, the MicroSp
10 # micro speech module for
11 # user can override both
12 # MicroSpeech(preprocesso
13 speech = MicroSpeech()
14
15 # Starts the audio stream
16 # If a callback is passed
17 # is detected. Alternativ
18 # returns if the timeout
19 speech.listen(callback=ca
```

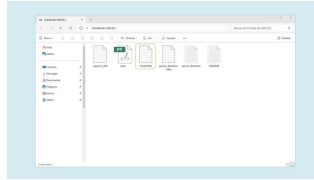
In the example from above you can notice that there is no model defined explicitly, this is because it will use the default built-in model pre-trained to recognize the **yes** and **no** keywords.

You can run the script and say the keywords, if any is recognized, the *Serial Terminal* will print the heard word and the inference scores.

Custom Speech Recognition Model

You can easily run custom speech recognition models also. To show you how, we are going to replicate the **yes** and **no** example but this time using the `.tflite` model file.

First, download the `.tflite` **model**



Speech recognition model
directory

Copy and paste the following script
based in the original example:

```
1 import time
2 import ml
3 from ml.apps import Micro
4
5 labels = ["Silence", "Unk
6
7 def callback(label, score
8     print(f'\nHeard: "{la
9
10 speech = MicroSpeech(micr
11
12 speech.listen(callback=ca
```

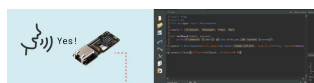
As you can see, there are some
differences between the original
example from which we can highlight
the following:

The `ml` module was imported

A labels list was created
including the model labels in a
specific order

The `MicroSpeech()` function
has been populated with the
model and labels list as
arguments.

Now, just say `yes` or `no` and you
will see the inference result in the
OpenMV Serial Terminal just as with
the original example.



Speech recognition example



If you want to create a custom model `.tflite` file, you can do it with your own keywords or sounds using [Edge Impulse](#).

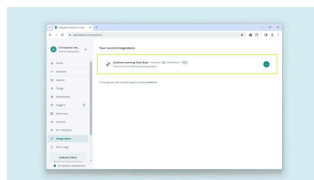
Machine Learning Tool

The main features of the Portenta Vision Shield are the audio and video capabilities. This makes it a perfect option for almost infinite machine-learning applications.

Creating this type of application has never been easier thanks to our Machine Learning Tool powered by Edge Impulse®, where we can easily create in a **No-Code** environment, **Audio**, **Motion**, **Proximity** and **Image** processing models.

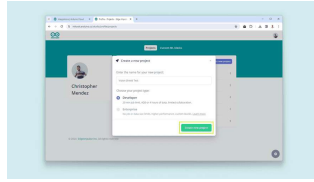
The first step to start creating awesome artificial intelligence and machine learning projects is to create an [Arduino Cloud](#) account.

There you will find a dedicated integration called **Machine Learning Tools**.



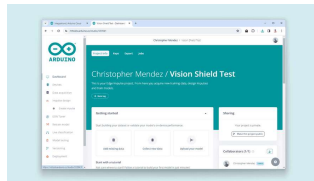
Machine Learning Tools on
Arduino Cloud

Once in, create a new project and



Creating a new project

Enter your newly created project and the landing page will look like the following:



Vision Shield project page

Edge Impulse® Environment Setup

Now, it is time to set up the **Edge Impulse®** environment on your PC. For this, follow [these](#) instructions to install the **Edge Impulse CLI**.



For Windows users: make sure to install [Visual Studio Community](#) and [Visual Studio Build Tools](#).

Download and install the latest **Arduino CLI** from [here](#). ([Video Guide for Windows](#))

Download the [latest Edge Impulse® firmware](#) for the Portenta H7, and unzip the file.

Open the flash script for your operating system (

`flash_windows.bat` ,

`flash_mac.command` or

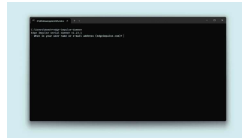
`flash_linux.sh`) to flash the

firmware

To test if the **Edge Impulse CLI** was installed correctly, open the **Command Prompt** or your favorite terminal and run:

```
edge-impulse-daemon
```

If everything goes okay, you should be asked for your Edge Impulse account credentials.



Edge Impulse
Daemon

Enter your account username or e-mail address and your password.

Select the project you have created on the Arduino ML Tools, it will be listed.

Give your device a name and wait for it to connect to the platform.

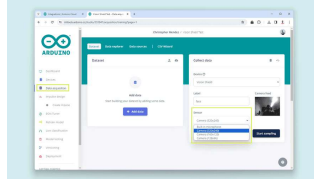


H7 + Vision Shield
correctly connected
to ML Tools

Uploading Sensor Data

The first thing to start developing a machine learning project is to create a *dataset* for your model. This means, uploading *data* to your model from the Vision Shield sensors.

To upload data from your Vision Shield on the Machine Learning Tools



Data Acquisition section

In this section, you will be able to select the Vision Shield onboard sensors individually.

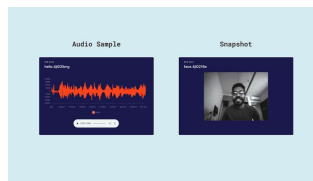
This is the supported sensors list:

Built-in microphone

Camera (320x240)

Camera (160x160)

Camera (128x96)



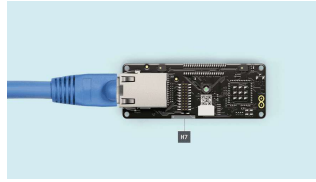
Samples uploaded using
the Vision Shield

Now you know how to start with our **Machine Learning Tools** creating your dataset from scratch, you can get inspired by some of our ML projects listed below:

[Image Classification with Edge Impulse®](#) (Article).

Ethernet (ASX00021)

The **Portenta Vision Shield - Ethernet** gives you the possibility of connecting your Portenta H7 board to the internet using a wired connection.



Ethernet cable connected

First, connect the Vision Shield - Ethernet to the Portenta H7. Now connect the USB-C® cable to the Portenta H7 and your computer. Lastly, connect the Ethernet cable to the Portenta Vision Shield's Ethernet port and your router or modem.

Now you are ready to test the connectivity with the following MicroPython script. This example lets you know if an Ethernet cable is connected successfully to the shield.

```
1 import network
2 import time
3
4 lan = network.LAN()
5
6 # Make sure Eth is not in
7 lan.config(low_power=False)
8
9 # Delay for auto negotiat
10 time.sleep(3.0)
11
12 while True:
13     print("Cable is", "co
14     time.sleep(1.0)
```

If the physical connection is detected, in the OpenMV Serial Monitor, you will see the following message:

```
Cable is connected.
```

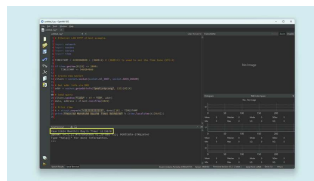
Once the connection is confirmed, we can try to connect to the internet using the example script below.

```

1 import network
2 import socket
3 import struct
4 import time
5
6 TIMESTAMP = 2208988800 +
7
8 if time.gmtime(0)[0] == 2
9     TIMESTAMP += 94668480
10
11 # Create new socket
12 client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
13
14 # Get addr info via DNS
15 addr = socket.getaddrinfo('www.google.com', 80)
16
17 # Send query
18 client.sendto("\x1b" + 47 * '\0', address = client.getsockname())
19 data, address = client.recv(1024)
20
21 # Print time
22 t = struct.unpack(">IIII", data)
23 print("Year:%d Month:%d Day:%d" % (t[0], t[1], t[2]))

```

Run the script and the current date and time will be printed in the OpenMV IDE Serial Monitor.



Ethernet connection
example script

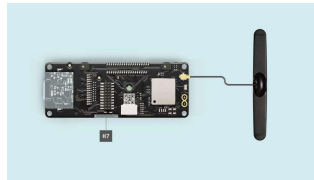


If you want to learn more, check the other Ethernet examples in the OpenMV IDE.

LoRa® (ASX00026)

The **Vision Shield with LoRa®** technology can extend our project

module for long-range communication in remote areas with a lack of internet access. Powered by the Murata CMWX1ZZABZ module which contains an STM32L0 processor along with a Semtech SX1276 Radio.



Antenna connection for LoRa® connectivity

To test the LoRa® connectivity, first, connect the Vision Shield with LoRa technology to the Portenta H7. Attach the antenna for LoRa connectivity to its respective connector. Now connect the USB-C® cable to the Portenta H7 and your computer.



Follow this [guide](#) to learn how to set up and create your **end device** on The Things Network.

Important hardware LoRa® configurations are listed below:

Setting	Compatibility
LoRa®-based MAC Version	V1.0.2
Class	A or C

The following MicroPython script lets you connect to The Things Network using LoRa® technology and send a `Hello World` message to it.


```

1 from lora import *
2
3 lora = Lora(band=BAND_AU915)
4
5 print("Firmware:", lora.firmware)
6 print("Device EUI:", lora.device_eui)
7 print("Data Rate:", lora.data_rate)
8 print("Join Status:", lora.join_status)
9
10 # Example keys for connection
11 appEui = "XXXXXXXXXXXX"
12 appKey = "XXXXXXXXXXXX"
13
14 try:
15     lora.join_OTAA(appEui, appKey)
16     # Or ABP:
17     # lora.join_ABP(devAddr, devKey, netId)
18 # You can catch individual errors
19 except LoraErrorTimeout:
20     print("Something went wrong")
21     print("ErrorTimeout")
22 except LoraErrorParam as e:
23     print("ErrorParam:", e)
24
25 print("Connected.")
26 lora.set_port(3)
27
28 try:

```

Find the frequency used in your country for **The Things Network** on this [list](#) and modify the parameter in the script within the following function.

```

1 lora = Lora(band=BAND_AU915)

```

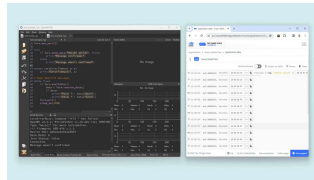
Define your application `appEUI` and `appKey` in the MicroPython script so the messages are correctly authenticated by the network server.

```

1 appEui = "XXXXXXXXXXXX"
2 appKey = "XXXXXXXXXXXX"

```

After configuring your credentials and frequency band, you can run the script. You must be in an area with LoRa®-based network coverage, if not, you should receive an alert from the code advising you to move near a window.



LoRa®-based network
uplink received on TTN



You can set up your own
LoRa®-based network using our
[LoRa® gateways](#)

Support

If you encounter any issues or have questions while working with the Vision Shield, we provide various support resources to help you find answers and solutions.

Help Center

Explore our [Help Center](#), which offers a comprehensive collection of articles and guides for the Vision Shield. The Arduino Help Center is designed to provide in-depth technical assistance and help you make the most of your device.

[Vision Shield Help Center
page](#)

Forum

users, share your experiences, and ask questions. The forum is an excellent place to learn from others, discuss issues, and discover new ideas and projects related to the Vision Shield.

[Vision Shield category in the Arduino Forum](#)

Contact Us

Please get in touch with our support team if you need personalized assistance or have questions not covered by the help and support resources described before. We're happy to help you with any issues or inquiries about the Vision Shield.

[Contact us page](#)

Trademark

Acknowledgments

LoRa® is a registered trademark of Semtech Corporation.

**Suggest
changes**

The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page [here](#).

**Need
support?**

[Help Center](#)
[Ask the Arduino Forum](#)
[Discover Arduino Discord](#)

License

The Arduino documentation is licensed under the [Creative Commons Attribution-Share Alike 4.0](#) license.

Was this article helpful?